

// C++ → UNREAL ENGINE 5.8 · POCKET EDITION

UNREAL 101 Cheat-Sheet

Type it. Break it. Fix it. Ship it.



World 1 · Terminal Town (C++)



TYPES

Every value has a type

```
int hp = 100;  double speed = 2.5;
bool alive = true;
std::string name = "Orby";
const int MAX_HP = 100;
```

FLOW

Decide and loop

```
if (hp <= 0) { alive = false; }
while (alive) { /* a turn */ }
for (int i = 0; i < 3; ++i) { }
```

FUNCTIONS

Functions

```
int RollDice(int sides) {
  return 1 + rand() % sides;
}
```

VECTOR

Lists that know their type

```
std::vector<int> bag = {5, 8};
bag.push_back(13);
for (int item : bag) { }
```

REFERENCES MUST MEMORIZE

Copy or original?

```
void Heal(Fighter& f) { f.hp += 5; }
void Show(const Fighter& f); // no copy
void Oops(Fighter f) { }     // a COPY!
```

CLASS

Classes guard their data

```
class Hero {
public:
  Hero(int hp) : hp_(hp) {}
  int Hp() const { return hp_; }
private:
  int hp_;
};
```

POINTERS

Pointers, honestly

```
Fighter* t = &g;   // address of g
t->hp = 5;         // (*t).hp
Fighter* none = nullptr;
```

VIRTUAL

One interface, many monsters

```
struct Monster {
  virtual ~Monster() = default;
  virtual void Attack() = 0;
};
struct Goblin : Monster {
  void Attack() override { }
};
```

UNIQUE_PTR

Who owns this?

```
auto m = std::make_unique<Goblin>();
m->Attack();     // freed automatically
```

World 2 · Orb Arena (Unreal)



LOG + TYPES

Say it in the log

```
UE_LOG(LogTemp, Warning,
  TEXT("HP=%d %s"), Hp, *Name);
int32  float  FString  FName
```

ACTOR MUST MEMORIZE

Your first Actor

```
UCLASS()
class AOrb : public AActor {
  GENERATED_BODY()
public:
  virtual void Tick(float Dt) override;
};
```

UPROPERTY + UFUNCTION

Knobs for the editor

```
UPROPERTY(EditAnywhere, Category="Orb")
float Spin = 90.f;
UFUNCTION(BlueprintCallable)
void Collect();
```

COMPONENTS

Actors are built from parts

```
Mesh = CreateDefaultSubobject
  <UStaticMeshComponent>(TEXT("Mesh"));
RootComponent = Mesh;
Light->SetupAttachment(Mesh);
```

OVERLAP + CAST

Something touched me

```
Sphere->OnComponentBeginOverlap
  .AddDynamic(this, &AOrb::OnHit);
UFUNCTION()   // required!
void OnHit(...);
if (ACharacter* P =
    Cast<ACharacter>(Other))
```

TARRAY + TMAP

Unreal's containers

```
TArray<int32> Scores;
Scores.Add(10);   Scores.Num();
TMap<FString, int32> Pts;
Pts.Add(TEXT("gold"), 50);
```

SPAWN + TIMER

Things appear over time

```
GetWorld()->SpawnActor<AOrb>(
  OrbClass, Loc, FRotator::ZeroRotator);
GetWorldTimerManager().SetTimer(
  T, this, &AGM::Spawn, 2.f, true);
```

GARBAGE COLLECTION MUST MEMORIZE

Memory the Unreal way

```
UPROPERTY() AActor* Keep; // GC-safe
TWeakObjectPtr<AActor> Maybe;
if (IsValid(Ptr)) { }   Destroy();
```

DELEGATES

Your own events

```
UPROPERTY(BlueprintAssignable)
FOnScore OnScore;
OnScore.Broadcast(Score);
GM->OnScore.AddDynamic(
  this, &AHud::Show);
```

NAME TAGS

The first letter tells you what it is

A	Actor	AOrb
U	Object / Component	UStaticMeshComponent
F	struct / value type	FVector
E	enum	EOrbType
T	template / container	TArray
I	interface	IInteractable
b	bool variable	bAlive



Boss fights

Compiler yelling? Read the FIRST error only. Fix it. Rebuild.

Copy trap No & means a copy. Use Fighter& to change the original.

Log shows garbage FString in UE_LOG needs a * in front.

Editor ignores your handler AddDynamic functions need UFUNCTION().